

Experiment No. 9

Object Detection and Recognition using Deep Learning Models on Standard Image Datasets

```
!pip install ultralytics opencv-python matplotlib -q
```

```
46.7/46.7 kB 1.3 MB/s eta 0:00:00
1.5/1.5 MB 12.8 MB/s eta 0:00:00
53.2/53.2 kB 1.7 MB/s eta 0:00:00
88.7/88.7 kB 7.6 MB/s eta 0:00:00
```

```
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO
```

Creating new Ultralytics Settings v0.0.8 file ☒

View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'

Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see <https://docs.ultralytics.com>

```
model = YOLO("yolo11n.pt")
```

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11n.pt> to 'yolo11n.pt': 100% — 5.4

```
!wget -q https://ultralytics.com/images/bus.jpg -O bus.jpg
```

```
img = cv2.imread("bus.jpg")
img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```
plt.figure(figsize=(10,7))
plt.imshow(img)
plt.axis("off")
plt.show()
```



```
!pip install ultralytics opencv-python matplotlib -q
```

```
import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO
```

```
model = YOLO("yolo11n.pt")
```

```
!wget -q https://ultralytics.com/images/bus.jpg -O bus.jpg
```

```
results = model("bus.jpg")
```

```

result_img = results[0].plot()

plt.figure(figsize=(10,7))
plt.imshow(cv2.cvtColor(result_img, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()

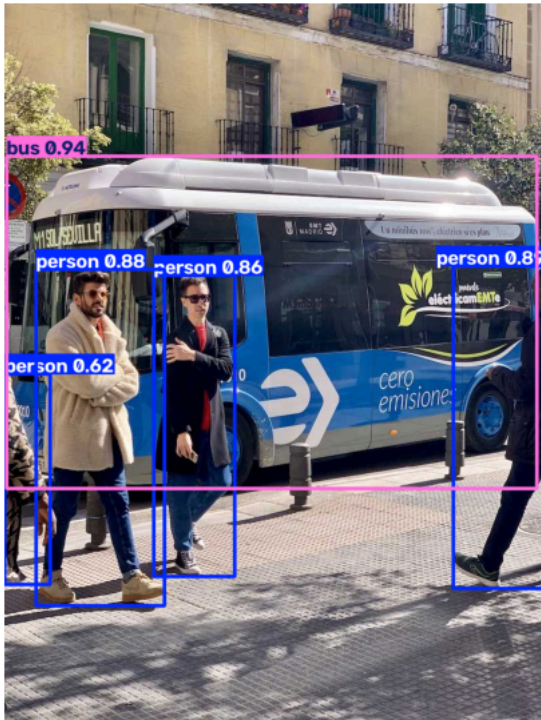
print("Detected Objects:")

for box in results[0].boxes:
    cls = int(box.cls[0])
    conf = float(box.conf[0])
    print(model.names[cls], round(conf, 2))

cv2.imwrite("detected_objects.jpg", result_img)
print("Result saved as detected_objects.jpg")

```

image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 403.2ms
Speed: 33.9ms preprocess, 403.2ms inference, 35.4ms postprocess per image at shape (1, 3, 640, 480)



Detected Objects:
bus 0.94
person 0.89
person 0.88
person 0.86
person 0.62
Result saved as detected_objects.jpg

```

for box in results[0].boxes:
    cls = int(box.cls[0])
    conf = float(box.conf[0])
    print(model.names[cls], round(conf, 2))

```

```

bus 0.94
person 0.89
person 0.88
person 0.86
person 0.62

```

```

cv2.imwrite("detected_objects.jpg", result_img)
print("Detection result saved successfully.")

```

Detection result saved successfully.

```
!pip install ultralytics opencv-python matplotlib -q
```

```

import cv2
import matplotlib.pyplot as plt
from ultralytics import YOLO

model = YOLO("yolo11n.pt")

```

```
!wget -q https://ultralytics.com/images/bus.jpg -O bus.jpg
```

```
results = model("bus.jpg")

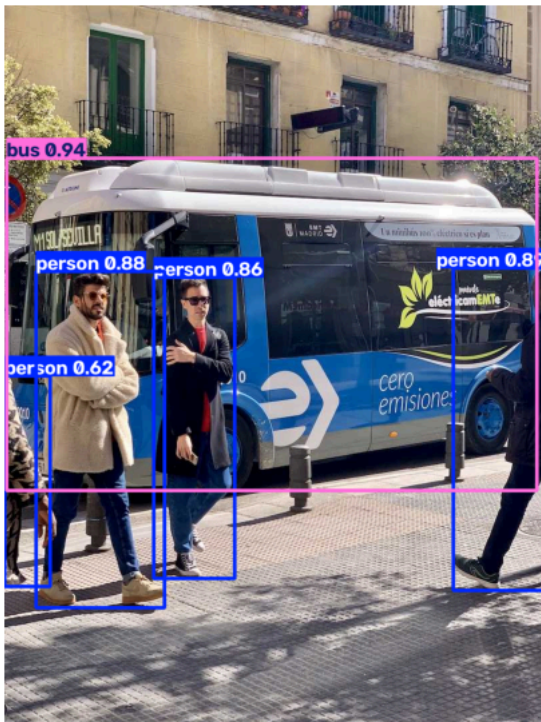
result_img = results[0].plot()

plt.figure(figsize=(10,7))
plt.imshow(cv2.cvtColor(result_img, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()

for box in results[0].boxes:
    cls = int(box.cls[0])
    conf = float(box.conf[0])
    print(model.names[cls], round(conf, 2))

cv2.imwrite("detected_objects.jpg", result_img)
```

image 1/1 /content/bus.jpg: 640x480 4 persons, 1 bus, 227.6ms
Speed: 8.2ms preprocess, 227.6ms inference, 1.7ms postprocess per image at shape (1, 3, 640, 480)



bus 0.94
person 0.89
person 0.88
person 0.86
person 0.62
True

Questions

1. Differentiate between image classification, object detection, and image segmentation.

Classification: Identifies the object class. Object detection: Identifies objects and draws bounding boxes. Segmentation: Classifies each pixel of an image.

2. Explain YOLO.

YOLO (You Only Look Once) detects objects in a single pass through a neural network. It predicts bounding boxes, class labels, and confidence scores

3. Compare YOLO, SSD, and Faster R-CNN.

Model	Speed	Accuracy
YOLO	Very Fast	High
SSD	Fast	High
Faster R-CNN	Slower	High

4. What is a bounding box?

A rectangular box around an object that shows its location and size.

5. What are confidence score and IoU?

Confidence score indicates prediction certainty. IoU measures overlap between predicted and actual bounding boxes.

6. Advantages of pre-trained models?

They save training time, require less data, and provide good accuracy.

7. Why are MS COCO and Pascal VOC used?

They contain labeled images and bounding-box annotations for training and evaluating object detection models.

8. Five applications of object detection.

Autonomous vehicles Surveillance Face detection Medical imaging Traffic monitoring

9. Challenges in object detection.

Poor lighting, occlusion, complex backgrounds, different object sizes, and overlapping objects.

10. How can object detection be improved?

Using better datasets, data augmentation, transfer learning, advanced models, and GPU accelerators